

Report conversation

as best practise, should you import with .js suffix

In JavaScript, whether to include the `.js` suffix when importing modules depends on the specific syntax and context of your code. Here are some guidelines:

1. Importing ES6 Modules (recommended):

If you are using ES6 module syntax to import modules, you should not include the `.js` suffix. Here's how you typically import modules in ES6:

javascript

 Copy code

```
import moduleName from './modulePath';
```

In this case, `modulePath` should point to the JavaScript module file (without the `.js` extension). The browser or Node.js environment will automatically look for the module and append `.js` if necessary.

2. Importing CommonJS Modules (Node.js):

In Node.js, when using CommonJS modules (e.g., `require`), you don't include the `.js` extension either. Node.js will resolve and load the module correctly. For example:

javascript

 Copy code

```
const moduleName = require('./modulePath');
```

3. Importing External Libraries:

When importing third-party libraries or packages, you usually specify the module name without the `.js` extension. For example:

javascript

 Copy code

```
import axios from 'axios'; // No '.js' extension
```

4. Importing Other File Types:

If you are importing non-JavaScript files, such as JSON, CSS, or other assets, you might need to include the file extension. For instance:

javascript

```
import jsonData from './data.json';  
import './styles.css';
```

In summary, including the `.js` suffix when importing JavaScript modules is generally unnecessary and not recommended in modern JavaScript development, especially when using ES6 module syntax. However, when dealing with non-JavaScript files, you should include the appropriate file extension. Always follow the guidelines and conventions of the module system you are using (e.g., ES6, CommonJS) and the tools or environments (browser, Node.js) you are working with.